Offline Installation & Configuration Guide

Infrastructure software for the Collateral Appraisal System — air-gapped (no internet) deployment

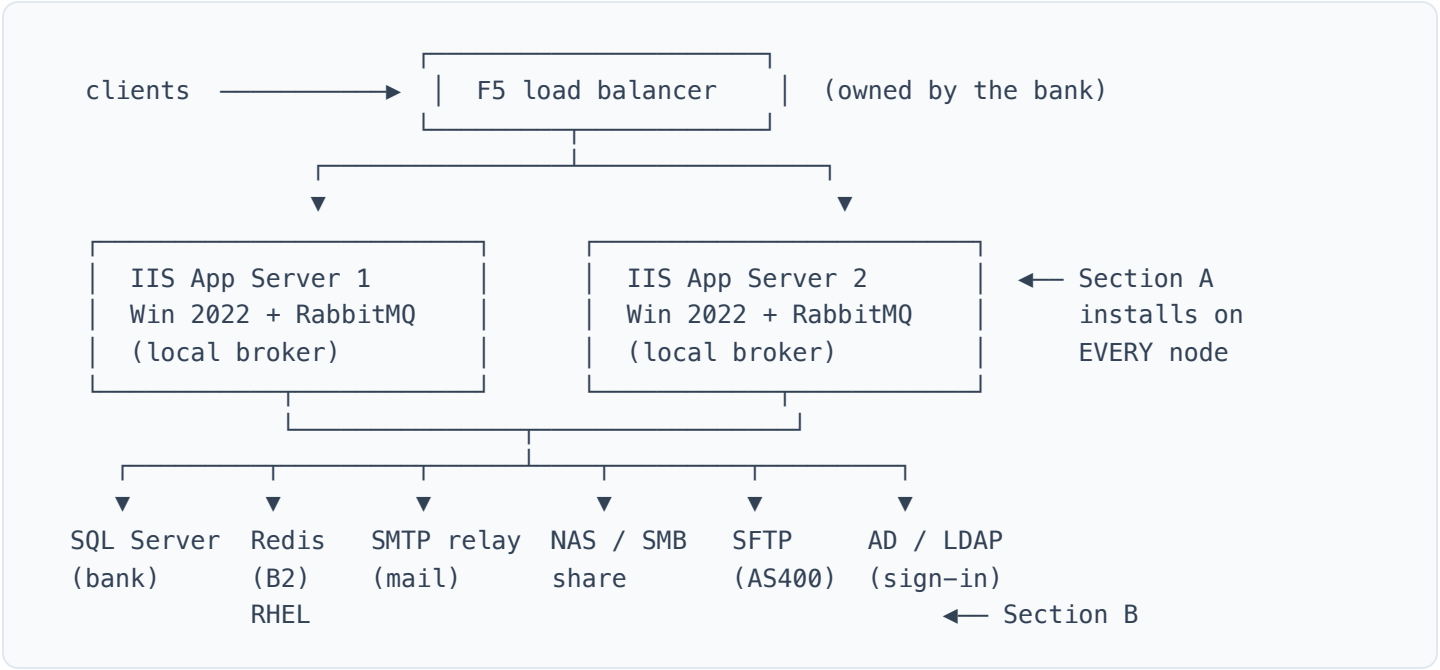
Audience: Bank Windows administrators, DBA, and Linux administrator · **Companion to:** `windows-iis-prerequisites.html` (the component list) · **Component IDs** (A1–A5, B2) match that document.

> **What this guide covers — and what it does not.** This runbook explains how to **install and configure the third-party software** that hosts the application, on a network with **no internet access**. It deliberately **excludes deployment of the application itself** (publishing the build, creating the IIS site, and filling in `appsettings.Production.json`) — that is a separate document. Where an install step produces a value the application later needs (e.g. the Edge executable path, the RabbitMQ credentials, the Redis host & password), record it on the sign-off sheet and hand it to whoever does the application deploy.

Contents

Target topology

Production runs as **two Windows Server 2022 / IIS application servers** behind a bank-owned **F5 load balancer**. Each app server hosts the application **plus its own RabbitMQ** broker (co-located, not shared). The two nodes share one **SQL Server**, one **Redis** server (on RHEL Linux), an SMTP relay, a NAS file share, an SFTP endpoint and the AD/LDAP directory. Because there are two app nodes (N=2), every shared component must be reachable from **both** nodes.



This guide details **A1–A5** (every Windows node) and **B2 (Redis on RHEL)**. SQL Server, SMTP, NAS, SFTP and AD are existing bank-provided services the application merely connects to — no install or configuration is documented for them here.

0. Offline staging workflow (read first — applies to every step)

None of the target servers can reach the internet, a package repository, WSUS, or Microsoft Web Platform Installer. Every installer must therefore be brought in by hand. Use this same pattern for **every** download referenced below:

1. **Transfer via the bank's approved media / file-transfer process** (sealed USB, one-way transfer gateway, etc.) — bring in each file named in Appendix A — Staging manifest.
2. **Copy to a local install folder** on the target server — `D:\CAS Installation` on each Windows node, `/opt/staging/` on the Linux node — then install locally from there.

Section A — On EACH Windows Server 2022 app node (repeat all steps on both nodes)

Perform A1 through A5 on **App Server 1**, then repeat identically on **App Server 2**. The two nodes are configured the same way; only the values that are intentionally shared (SQL, Redis, NAS) point at common servers. Run all steps from an **elevated Windows PowerShell** (Run as Administrator), with the staged files already copied to `D:\CAS Installation` per Section 0.

A1 IIS (Web Server) role + features — offline

The application is hosted **in-process inside IIS**. Install the Web Server role plus the features the app needs.

```
Install-WindowsFeature -Name `
  Web-Server, Web-WebServer, Web-Common-Http, Web-Static-Content, Web-Default-Doc, `
  Web-Http-Errors, Web-Health, Web-Http-Logging, `
  Web-Performance, Web-Stat-Compression, `
  Web-Security, Web-Filtering `
  -IncludeManagementTools
```

Verify: `Get-WindowsFeature Web-Server` shows `Install State : Installed`. Browsing to `http://localhost/` shows the default IIS welcome page.

A2 .NET 9 Hosting Bundle — offline

Install the **Hosting Bundle** — not the plain .NET runtime. The Hosting Bundle adds the .NET Runtime + ASP.NET Core Runtime **and registers the ASP.NET Core Module v2 (ANCM)** in IIS, which is what lets IIS run the app in-process. **Install A1 (IIS) first**, otherwise ANCM is not registered and you must re-run the bundle.

```
cd "D:\CAS Installation"
.\dotnet-hosting-9.0.11-win.exe /install /quiet /norestart
# after it completes, recycle IIS so the new native module loads
iisreset
```

Verify:

```
dotnet --list-runtimes      # lists Microsoft.AspNetCore.App 9.0.11 and Microsoft.NETCore
(Get-WebGlobalModule).Name | Select-String AspNetCore   # shows AspNetCoreModuleV2
```

A3 IIS URL Rewrite Module — offline

URL Rewrite supports HTTPS-redirect and forwarded-header rules and is expected in most bank IIS setups. The F5 is the only proxy in front, so URL Rewrite alone is sufficient (no ARR needed). The normal installer (Web PI) needs the internet, so install the standalone MSI:

```
cd "D:\CAS Installation"
msiexec /i rewrite_amd64_en-US.msi /qn /norestart
iisreset
```

Verify: open **IIS Manager** → select the server node → the Features view shows a **URL Rewrite** icon.

A4 Microsoft Edge for PDF rendering — configure only

The Reporting module renders PDFs with headless Chromium (PuppeteerSharp), which drives a Chromium-based browser over the DevTools protocol. **Microsoft Edge ships in-box on Windows Server 2022**, so there is **nothing to install** — you only confirm the executable path and hand it to the application-deploy step.

```
$edge = "C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe"
Test-Path $edge        # expect: True
& $edge --version       # prints the installed Edge/Chromium version
```

A5 Erlang/OTP + RabbitMQ — offline

RabbitMQ is the message broker for the application's event bus. **Each app node runs its own local broker — there is no clustering.** RabbitMQ on Windows runs on the Erlang runtime, so install **Erlang first, then RabbitMQ**, and they must be a **compatible pair**.

1. **Install Erlang/OTP** (silent), then set the `ERLANG_HOME` system variable RabbitMQ looks for:

   ```
   cd "D:\CAS Installation"
   .\otp_win64_28.5.0.2.exe /S
   [Environment]::SetEnvironmentVariable("ERLANG_HOME", "C:\Program Files\Erlang OTP", "Machi
   ```

   Adjust the path to the actual Erlang install directory (the installer prints it).

2. **Install RabbitMQ** (silent). It registers and starts as a Windows service automatically:

   ```
   .\rabbitmq-server-4.2.6.exe /S
   ```

3. **Sync the Erlang cookie** so `rabbitmqctl` can authenticate to the broker. The RabbitMQ **service** runs as **LocalSystem** and reads its cookie from the system profile, but `rabbitmqctl` launched in your admin shell reads the cookie from **your** profile — if they differ you get "authentication failed / unable to connect to node". Copy the service's cookie to your profile so they match:

   ```
   # compare the two cookies (they must be identical)
   Get-Content "$env:SystemRoot\System32\config\systemprofile\.erlang.cookie"
   Get-Content "$env:USERPROFILE\.erlang.cookie"

   # if they differ (or yours is missing), copy the service cookie over yours
   Copy-Item "$env:SystemRoot\System32\config\systemprofile\.erlang.cookie" `
     "$env:USERPROFILE\.erlang.cookie" -Force
   ```

   Then confirm connectivity from the `sbin` folder: `.\rabbitmqctl.bat status` should report the running node (not a cookie/auth error) before continuing.

4. **Enable the management plugin** (admin UI + `rabbitmqctl` niceties). Run from the RabbitMQ `sbin` folder:

```
cd "C:\Program Files\RabbitMQ Server\rabbitmq_server-4.2.6\sbin"
.\rabbitmq-plugins.bat enable rabbitmq_management
```

5. **Create the application user + virtual host and remove the default `guest`** account (it is a well-known credential and must not survive):

```
.\rabbitmqctl.bat add_user <RABBITMQ_USER> <RABBITMQ_PASSWORD>
.\rabbitmqctl.bat set_user_tags <RABBITMQ_USER> administrator
.\rabbitmqctl.bat set_permissions -p / <RABBITMQ_USER> ".*" ".*" ".*"
.\rabbitmqctl.bat delete_user guest
```

   The application connects to its own node at `amqp://localhost:5672/` (default virtual host `/`) using these credentials — record them for the app-deploy step.

Verify: `.\rabbitmqctl.bat status` reports the running node; `.\rabbitmq-plugins.bat list` shows `[E*] rabbitmq_management`; the UI answers at `http://localhost:15672` (log in with the new user, confirm `guest` is gone).

Per-node completion checklist

- ☐ A1 — IIS Web Server role installed; default page loads.
- ☐ A2 — .NET 9 Hosting Bundle installed; `AspNetCoreModuleV2` present; `iisreset` run.
- ☐ A3 — URL Rewrite installed.
- ☐ A4 — Edge path `...\msedge.exe` confirmed and recorded for the app config.
- ☐ A5 — Erlang + RabbitMQ installed; management plugin on; app user created; `guest` deleted.
- ☐ Repeated identically on the second app node.

Section B — Backing infrastructure

B2 Redis on RHEL Linux — offline, build from source

Redis provides the distributed cache and — critically for the two-node setup — the **SignalR backplane**, so real-time messages reach a browser no matter which app node it is connected to. **Both app nodes must reach this one Redis.** The primary, most reliable offline method on an air-gapped RHEL box is to **build from the official source tarballs** (jemalloc + Redis), which depend only on the local C toolchain.

1. **Confirm a build toolchain is present** (Redis only needs a C compiler + make; `tcl` is needed only for the optional test suite):

```
gcc --version && make --version       # both must already be installed
```

> **If `gcc` / `make` are absent — install them from the RHEL ISO (no internet needed).** The RHEL installation ISO already contains the toolchain in its `BaseOS` and `AppStream` repositories. Mount the ISO, point `dnf` at it as a local repo, then install:
>
> ```
> # 1) mount the RHEL ISO (attached as a virtual DVD drive)
> sudo mkdir -p /mnt/disc
> sudo mount /dev/sr0 /mnt/disc            # ISO file on disk instead? sudo mount -o loo
>
> # 2) define a local offline repo pointing at the mounted ISO
> sudo tee /etc/yum.repos.d/rhel9-local.repo <<'EOF'
> [BaseOS]
> name=RHEL 9 BaseOS (Offline)
> baseurl=file:///mnt/disc/BaseOS
> enabled=1
> gpgcheck=0
>
> [AppStream]
> name=RHEL 9 AppStream (Offline)
> baseurl=file:///mnt/disc/AppStream
> enabled=1
> gpgcheck=0
> EOF
>
> # 3) rebuild the dnf cache and confirm both repos are seen
> sudo dnf clean all
> sudo dnf makecache
> sudo dnf repolist
>
> # 4) install the build toolchain (gcc-c++ optional; tcl only for Redis' self-test)
> sudo dnf install -y gcc gcc-c++ make
> ```

2. **Build and install jemalloc** (Redis' preferred memory allocator). On a minimal RHEL image jemalloc is often not present; build it from source staged per Section 0:

```
sudo mkdir -p /opt/staging && cd /opt/staging
# jemalloc-5.3.0.tar.bz2 already copied here via approved media
tar xjf jemalloc-5.3.0.tar.bz2
cd jemalloc-5.3.0

# If ./configure is MISSING (you took the GitHub source ZIP / git clone, not the
# release tarball), generate it first with autogen.sh — it needs the autotools:
#   sudo dnf install -y autoconf automake libtool   # from the same RHEL ISO repo
#   ./autogen.sh

./configure
make
sudo make install
sudo ldconfig                           # refresh the linker cache so libjemalloc is found
```

The jemalloc **release tarball** (`jemalloc-5.3.0.tar.bz2` on the GitHub Releases page) already ships a generated `configure` — run it directly. Only a **git checkout / source ZIP** lacks `configure` and needs `./autogen.sh` first (which is why it ran last time).

3. **Stage and unpack Redis** (after transfer per Section 0):

```
cd /opt/staging
# redis-stable.tar.gz already copied here via approved media
tar xzf redis-stable.tar.gz
cd redis-stable
```

4. **Build and install** into `/usr/local`, forcing the jemalloc allocator:

```
make MALLOC=jemalloc
sudo make install PREFIX=/usr/local      # installs redis-server, redis-cli to /usr/local/b
```

`MALLOC=jemalloc` is the correct, current Redis build flag (the legacy `USE_JEMALLOC=yes` is broken and may silently fall back to glibc malloc). Verify the allocator afterwards with `redis-server --version` / `INFO memory` (`mem_allocator:jemalloc`).

5. **Create a service account, config and data directories:**

```
sudo useradd -r -s /sbin/nologin redis
sudo mkdir -p /var/lib/redis /etc/redis
sudo chown -R redis:redis /var/lib/redis /etc/redis
sudo chmod 750 /var/lib/redis
```

`-r` makes a no-login **system** account; `chmod 750` keeps the data dir private to the `redis` user. Logging goes to **journald** (see `logfile ""` below), so no `/var/log/redis` is needed.

6. **Create** `/etc/redis/redis.conf`. Tuned for a **cache + SignalR backplane** (transient data, no disk persistence). Bind to the interface the app nodes reach, require a password, and keep protected mode on:

```
sudo tee /etc/redis/redis.conf >/dev/null <<'EOF'
# Network — listen on all interfaces (firewall + requirepass restrict access)
bind 0.0.0.0
protected-mode yes
port 6379
tcp-keepalive 300

# Security
requirepass <REDIS_PASSWORD>

# Process control (systemd-managed)
daemonize no
supervised systemd

# Data directory + log (empty logfile = stdout -> journald)
dir /var/lib/redis
logfile ""

# Memory — cache + backplane; adjust maxmemory to the server's RAM
maxmemory 4gb
maxmemory-policy allkeys-lru

# Persistence OFF — data is a cache / SignalR backplane (transient)
save ""
appendonly no

# Pub/Sub — don't disconnect slow SignalR subscribers (bounded, not unlimited)
client-output-buffer-limit pubsub 256mb 64mb 60
EOF

# lock the config down — it holds the password
sudo chown redis:redis /etc/redis/redis.conf
sudo chmod 640 /etc/redis/redis.conf
```

SignalR-backplane notes: `allkeys-lru` is safe — the backplane is pub/sub and stores no keys, so eviction never touches it. `save ""` + `appendonly no` disable disk persistence (not needed for transient cache/backplane). The `pubsub` buffer is raised (not `0 0 0`) so a slow subscriber won't be dropped, while still protecting Redis from a runaway client.

7. **Create the systemd unit** `/etc/systemd/system/redis.service`:

```
sudo tee /etc/systemd/system/redis.service >/dev/null <<'EOF'
[Unit]
Description=Redis In-Memory Data Store (Collateral Appraisal System)
After=network.target

[Service]
User=redis
Group=redis
ExecStart=/usr/local/bin/redis-server /etc/redis/redis.conf
Restart=always
LimitNOFILE=100000
TimeoutStopSec=30

[Install]
WantedBy=multi-user.target
EOF
```

No `ExecStop` is needed: systemd's default stop sends **SIGTERM**, which `redis-server` handles as a clean shutdown — so we avoid putting the password in the unit file (which an authenticated `redis-cli ... shutdown` would require). `LimitNOFILE=100000` raises the file-descriptor cap Redis needs for many client connections. then enable and start it:

```
sudo systemctl daemon-reload
sudo systemctl enable --now redis
```

8. **Open the firewall** to the two app nodes only:

```
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
  source address="<APPNODE1_IP>" port port="6379" protocol="tcp" accept'
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
  source address="<APPNODE2_IP>" port port="6379" protocol="tcp" accept'
sudo firewall-cmd --reload
```

If SELinux is enforcing and you bind to a custom interface/port, also confirm Redis is allowed (`setsebool -P` / `semanage port` as your SELinux policy requires).

> **Connection string the app will use** (StackExchange.Redis format, recorded for the app-deploy step): `<REDIS_LAN_IP>:6379,password=<REDIS_PASSWORD>,abortConnect=false`. The same Redis serves both the cache and the SignalR backplane.

Verify:

```
sudo systemctl status redis                  # active (running)
sudo journalctl -u redis -n 50 --no-pager    # Redis startup log (via journald)
# on the Redis host
redis-cli -a <REDIS_PASSWORD> ping            # PONG
# from an app node (proves network + firewall + auth)
redis-cli -h <REDIS_LAN_IP> -a <REDIS_PASSWORD> ping   # PONG
```

Verification summary (sign-off)

#	Component	Check command	Expected
A1	IIS role	`Get-WindowsFeature Web-Server`	Install State : Installed; default page loads
A2	.NET 9 Hosting Bundle	`dotnet --list-runtimes`	`Microsoft.AspNetCore.App 9.0.11`; `AspNetCoreModuleV2` present
A3	URL Rewrite	IIS Manager → server Features view	URL Rewrite icon
A4	Edge (PDF)	`& "…\msedge.exe" --version`	Prints Edge version; path recorded
A5	RabbitMQ + Erlang	`rabbitmqctl status` / UI :15672	Node running; mgmt plugin on; `guest` deleted
B2	Redis	`redis-cli -h host -a … ping`	`PONG` from host AND from an app node

Appendix A — Staging manifest (the offline shopping list)

Bring each file across via the bank's approved media / file-transfer process. Versions shown are representative — confirm the current/approved version at staging time, and for A5 confirm the RabbitMQ↔Erlang pairing on the vendor compatibility matrix.

For	File (pattern)	Version
A2	`dotnet-hosting-9.0.11-win.exe` (ASP.NET Core Hosting Bundle)	9.0.11
A3	`rewrite_amd64_en-US.msi` (URL Rewrite)	2.1
A4	— none — Edge ships in-box on Server 2022	in-box
A5	`otp_win64_28.5.0.2.exe` (Erlang/OTP)	28.5.0.2
A5	`rabbitmq-server-4.2.6.exe`	4.2.6
B2	`jemalloc-5.3.0.tar.bz2` (source)	5.3.0
B2	`redis-stable.tar.gz` (source)	8.4
B2	(only if toolchain absent) `gcc`, `gcc-c++`, `make`	per RHEL ver.

Appendix B — Placeholder glossary

Replace every `<PLACEHOLDER>` in the commands above with the bank's real value. These are the values these installs produce or require — hand them to whoever performs the application deploy.

Placeholder	Meaning
`<APPNODE1_IP>` / `<APPNODE2_IP>`	IP addresses of the two IIS app servers (for firewall scoping)
`<RABBITMQ_USER>` / `<RABBITMQ_PASSWORD>`	Application's RabbitMQ credentials, created per node
`<REDIS_LAN_IP>`	Redis server IP on the interface the app nodes reach
`<REDIS_PASSWORD>`	Redis `requirepass` value